Product Inventory Management System

Project Documentation

- Team ID : SWTID-2026-1300
- Team Size : 4
- Team Leader : PRABU.R
- Team member : MURALIDHARAN.S
- Team member : PRAVEEN KUMAR M
- Team member : MOOHAMMAD THAMEEM ANSARI

Roles and Responsibility

✓ Member 1 : **PRABU. R (Team Leader)**
File configuration
Phase Documentation
✓ Member 2 : MURALIDHARAN. S
GitUp Upload
✓Member 3 – PRAVEEN KUMAR M
Demo Video
✓Member 4 – MOOHAMMAD THAMEEM ANSARI
 # PPT presentation

User

- **Registration:** Register using name, email and password
- **Login:** Login using registered credentials
- **Add Product:** Create product with name, price, quantity and description
- **Edit Product:** Update existing product
- **Delete Product:** Remove product from inventory

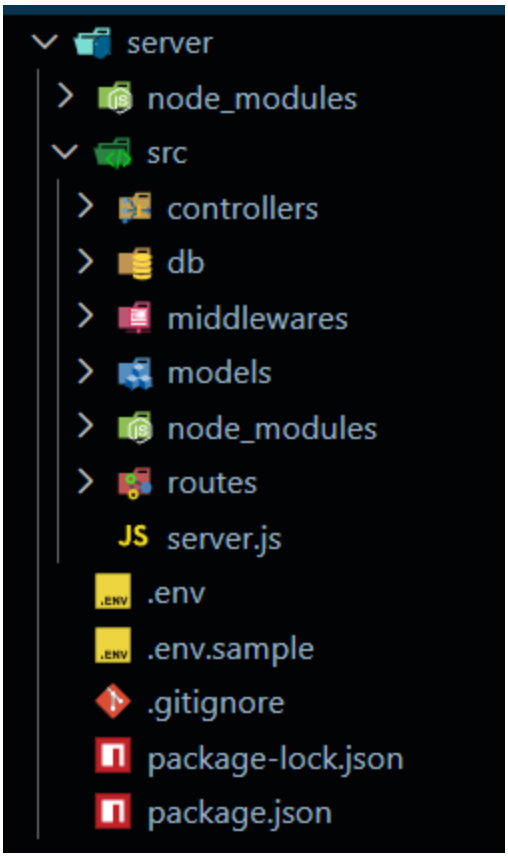
- **Search Product:** Search products using keyword

User Flow

Project Setup and Configuration

Folder Structure

```
server/
│
├── node_modules/
├── src/
│   ├── controllers/
│   ├── db/
│   ├── middlewares/
│   ├── models/
│   ├── routes/
│   └── server.js
│
├── .env
├── .gitignore
├── package-lock.json
└── package.json
```



Initialize Node.js Project:

Open terminal inside the server folder and run:

```
npm init -y
```

This creates the `package.json` file which manages project dependencies.

Install Required Packages:

Install backend dependencies:

```
npm install express mongoose cors dotenv jsonwebtoken bcryptjs
```

Package Purpose:

- express → Backend framework
- mongoose → MongoDB connection
- cors → Cross-origin requests
- dotenv → Environment variables
- jsonwebtoken → Authentication
- bcryptjs → Password hashing

Create Environment File:

Create a `.env` file in the root of the server folder.
Example:

```
PORT=5000
MONGO_URI=your_mongodb_connection_string
JWT_SECRET=your_secret_key
```

Backend Development

Setup Express Server:
Inside `src/server.js`, create the basic Express server.

```
const express = require('express')
const app = express()
const port = 3000

app.get('/', (req, res) => {
  res.send('Hello World!')
})

app.listen(port, () => {
  console.log(`Example app listening on port ${port}`)
})
```

Create Controllers
Inside: src/controllers/
Create:
auth.controller.js

```
export const registerUser = async (req, res) => {
  try {…
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};


export const loginUser = async (req, res) => {
  try {…
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};
```

Auth Controllers

The Auth Controller manages all user authentication-related operations in the system. It handles user registration and login functionality while ensuring secure access using JWT tokens.

Responsibilities:

- Register new users
- Validate user input data
- Hash passwords using bcrypt before storing
- Authenticate users during login
- Generate JWT tokens for authorized access
- Send appropriate success and error responses

Main Functionalities:

- Register User: Creates a new user account after validating email uniqueness and encrypting the password.
- Login User: Verifies credentials and returns a JWT token for protected API access.

product.controller.js

```
export const createProduct = async (req, res) => {
> try {···
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

export const getProducts = async (req, res) => {
> try {···
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

export const updateProduct = async (req, res) => {
> try {···
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

export const deleteProduct = async (req, res) => {
> try {···
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};
```

Product Controller

The **Product Controller** handles all product inventory operations and contains the business logic for managing products linked to individual users.

Responsibilities:

- Add new products to inventory
- Fetch products belonging to logged-in users
- Update product details like price, quantity, or description
- Delete products securely
- Perform product search functionality

Main Functionalities:

- **Create Product:** Adds a new product with a reference to the authenticated user.
- **Get Products:** Retrieves only the products created by the logged-in user.
- **Update Product:** Modifies product details while verifying ownership.
- **Delete Product:** Removes a product safely from the database.
- **Search Product:** Allows filtering products based on name.

Create Middleware

Inside: src/middlewares/

Create:auth.middleware.js

```
export const protect = (req, res, next) => {
  const authHeader = req.headers.authorization;

  if (!authHeader || !authHeader.startsWith("Bearer ")) {
    return res.status(401).json({ message: "Not authorized" });
  }

  try {
    const token = authHeader.split(" ")[1];
    const decoded = jwt.verify(token, process.env.JWT_SECRET);

    req.user = decoded;
    next();
  } catch (error) {
    res.status(401).json({ message: "Token invalid" });
  }
};
```

Auth Middleware (protect)

The **Auth Middleware** secures protected routes by verifying the JSON Web Token (JWT) sent in the request header. It checks for a valid `Authorization` header starting with `"Bearer "` and validates the token using the secret key. If the token is valid, the decoded user information is attached to `req.user` and the request proceeds to the next controller. If the token is missing or invalid, the middleware returns a **401 Unauthorized** response, preventing access to private APIs.

Create Routes

Inside: src/routes/

Create:

auth.routes.js

```
const authRoutes = express.Router();


authRoutes.post("/register", registerUser);

authRoutes.post("/login", loginUser);


export default authRoutes;
```

Auth Routes:
Auth routes handle user authentication by providing endpoints for registration and login. These routes connect incoming requests to the Auth Controller to create new users and generate JWT tokens for secure access.

product.routes.js

```
const productRouter = express.Router();

productRouter.post("/", protect, createProduct);
productRouter.get("/", protect, getProducts);
productRouter.get("/:id", protect, getProductById);
productRouter.put("/:id", protect, updateProduct);
productRouter.delete("/:id", protect, deleteProduct);

export default productRouter;
```

Product Routes:

Product routes manage all inventory-related operations such as creating, retrieving, updating, and deleting products. These routes are protected using authentication middleware to ensure only authorized users can access their own product data.

Connect Routes to Server

Import all routes into `server.js`

```
app.use("/api/v1/auth", authRoutes);
app.use("/api/v1/products", productRoutes);
```

These route configurations connect the authentication and product APIs to the main Express server using versioned endpoints. The `/api/v1/auth` path handles all user authentication requests, while `/api/v1/products` manages product-related operations, helping maintain a structured and scalable API architecture.

Database Configuration and Setup

- Create MongoDB Atlas cluster
- Add connection string to `.env`
- Connect DB before API testing.

Database Connection:

Create a file inside: src/db/db.js
Purpose: Connect backend to MongoDB using mongoose.

```javascript
import mongoose from "mongoose";

const connectDB = async () => {
    try {
        const connectionInstance = await mongoose.connect(`${process.env.MONGODB_URI}`)
        console.log(`MongoDB connected !! `);
    } catch (error) {
        console.log("MONGODB connection FAILED ", error);
        process.exit(1)
    }
}

export default connectDB
```

The connectDB function establishes a connection between the application and MongoDB using Mongoose and environment-based configuration. It ensures the server starts only after a successful database connection and logs an error message if the connection fails.

Create Models:

Inside `src/models/` create:

- User.js
- Product.js

User Model:

```javascript
const userSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: true,
      trim: true,
    },
    email: {
      type: String,
      required: true,
      unique: true,
      lowercase: true,
    },
    password: {
      type: String,
      required: true,
      minlength: 6,
    },
  },
  { timestamps: true }
);

export default mongoose.model("User", userSchema);
```

User Schema:

The User schema defines the structure for storing user details such as name, email, and hashed password in the database. It supports secure authentication and maintains timestamps for record tracking.

Product Model:

```
const productSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: true,
      trim: true,
    },

    price: {
      type: Number,
      required: true,
      min: 0,
    },

    quantity: {
      type: Number,
      required: true,
      min: 0,
      default: 0,
    },

    description: {
      type: String,
      default: "",
    },

    userId: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "User",
      required: true,
    },
  },
  { timestamps: true }
);

export default mongoose.model("Product", productSchema);
```

Product Schema:
The Product schema represents inventory items and includes fields like name, price, quantity, description, and a userId reference. It ensures each product is linked to its owner, enabling private data access and management.

Project Execution

Execution Steps

1. Open terminal in server folder.
2. Make sure you have a start script in the package.js file.
3. Run:

```
npm start
```

3. Console should display:

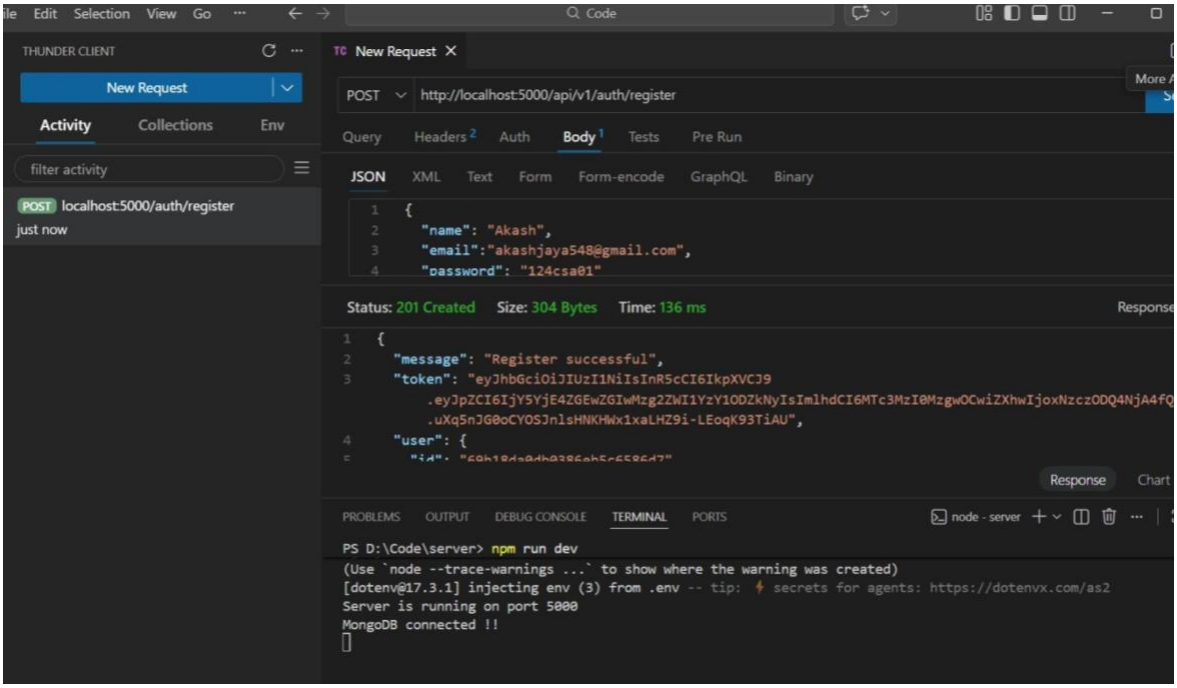
```
Server running on PORT
Database Connected
```

API Testing :

Use Thunder Client to test APIs:

Test APIs:

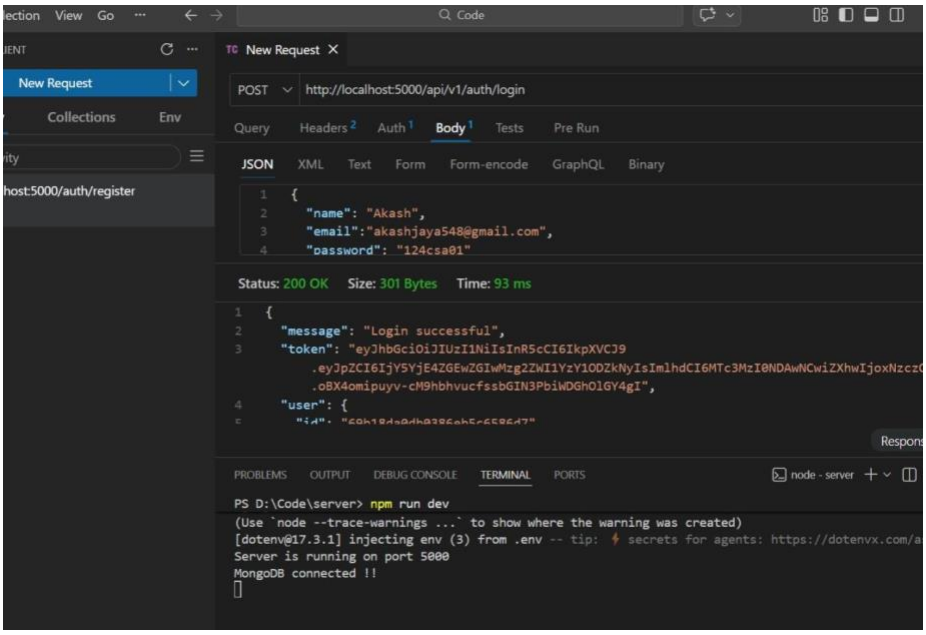
Register User:



Register:

The register functionality allows new users to create an account by submitting their details, which are validated and securely stored with a hashed password. It ensures unique user creation before granting access to the system.

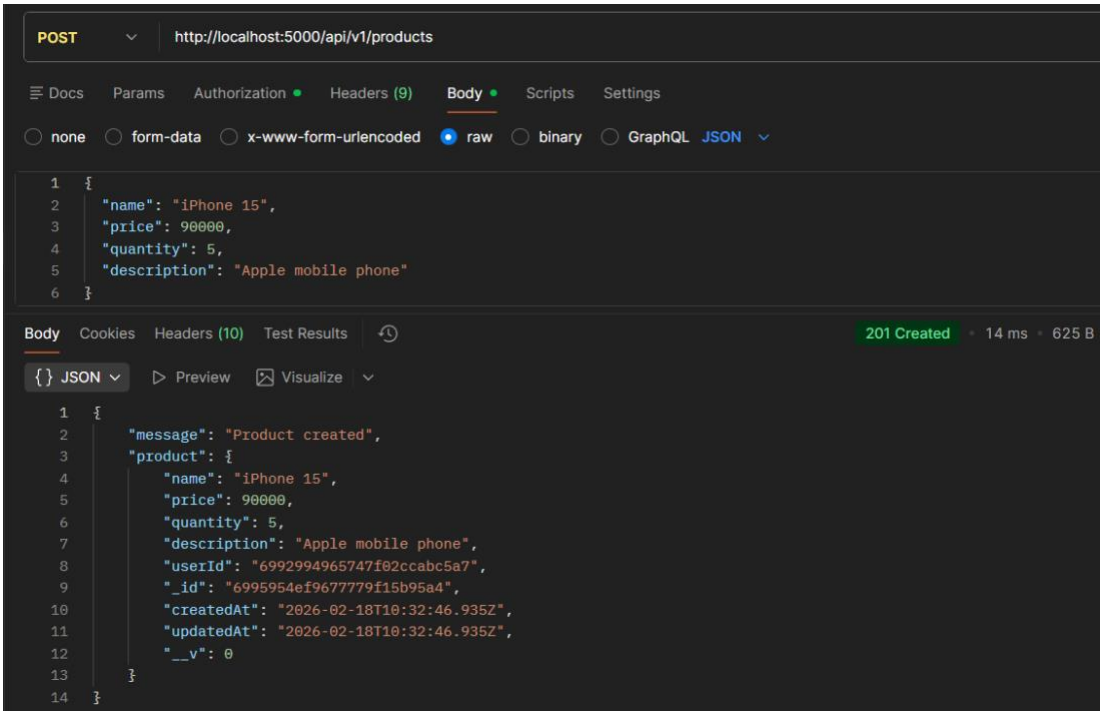
Login User



Login:

The login functionality verifies user credentials and generates a JWT token upon successful authentication. This token is used to access protected routes securely.

Create Product

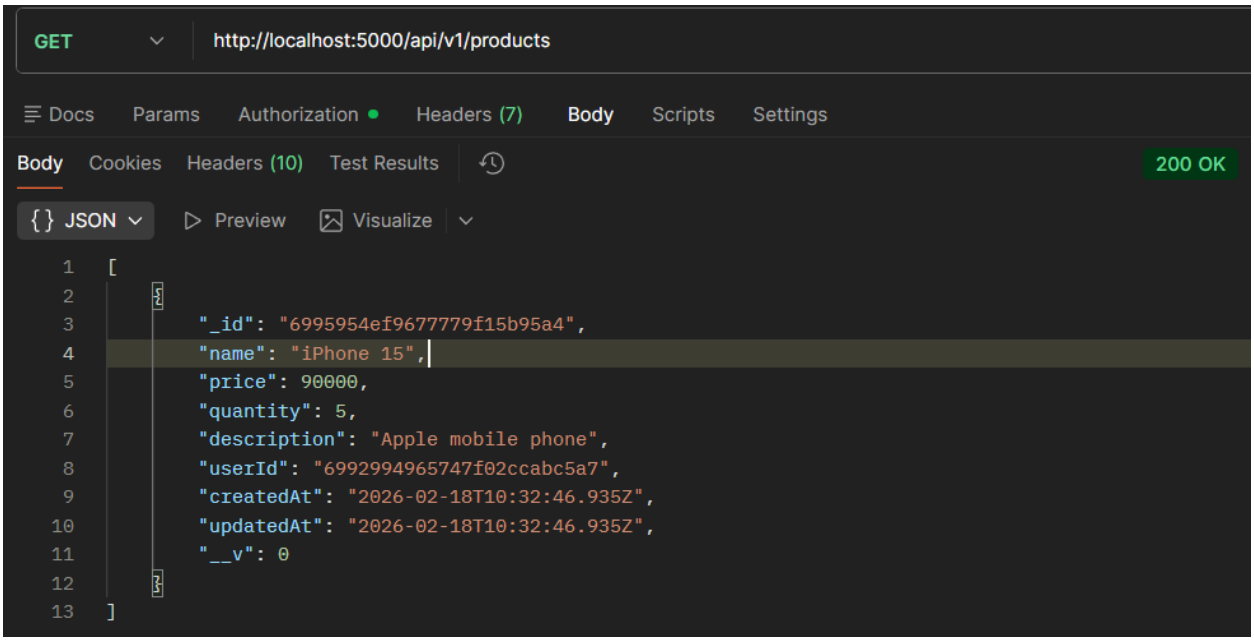


Create Product:

Allows an authenticated user to add a new product to their inventory by submitting product details.

The product is saved with the user's ID to maintain ownership.

Get all Products



Get all Products:

Fetches all products that belong to the logged-in user. It ensures users can view only their own inventory data.

Update Product

PUT http://localhost:5000/api/v1/products/6995954ef9677779f15b95a4

Docs Params Authorization Headers (9) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
{
  "name": "iPhone 19 Pro",
  "price": 95000,
  "quantity": 10,
  "description": "Updated phone"
}
```

Body Cookies Headers (10) Test Results 200 OK 37 ms

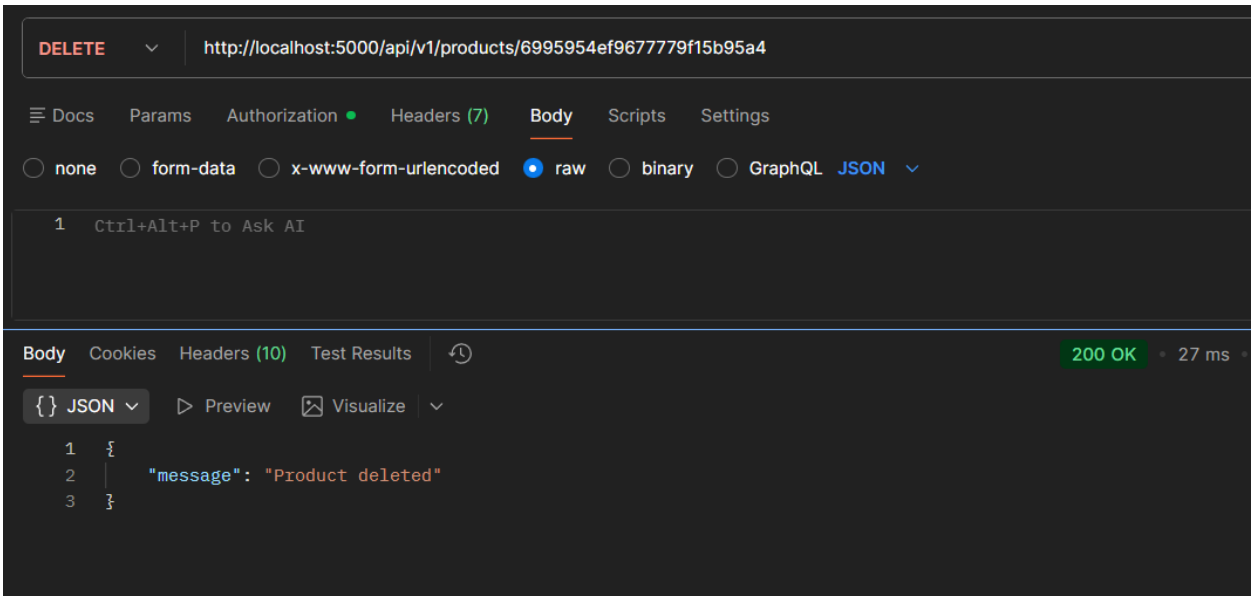
JSON Preview Visualize


```
{
    "message": "Product updated",
    "product": {
        "_id": "6995954ef9677779f15b95a4",
        "name": "iPhone 19 Pro",
        "price": 95000,
        "quantity": 10,
        "description": "Updated phone",
        "userId": "6992994965747f02ccabc5a7",
        "createdAt": "2026-02-18T10:32:46.935Z",
        "updatedAt": "2026-02-18T10:35:11.547Z",
        "__v": 0
    }
}
```


Update Product:

Enables users to modify existing product details such as name, price, or quantity. The system verifies ownership before allowing any updates.

Delete Product



Delete Product:

Allows users to permanently remove a product from their inventory. The operation is protected to ensure only the product owner can delete it.

Project Demo Video: [Nm project vd.mp4]